

Dev Legacy - Mission de maintenance & modernisation

Énoncé & consignes de rendu

1. Le contexte

Vous revenez sur votre projet **My Digital Start-up** après des mois. Le code est le vôtre... mais à moitié oublié, avec ses raccourcis et ses dépendances qui ont pris la poussière. Pire : un **rapport de sécurité** (`npm audit` / Dependabot) signale des failles dans vos librairies, et plusieurs sont en retard de **plusieurs versions majeures**. Il faut **maintenir et moderniser**, sans tout casser.

Aujourd'hui vous n'êtes plus en train de *créer* : vous êtes **mainteneur**. Vous devez livrer **comme un pro** : comprendre, sécuriser, mettre à jour, adapter, tester, documenter.

Ce qui est noté, ce n'est pas le volume de code, c'est votre **méthode pour intervenir sans rien casser** dans un projet existant ainsi que votre capacité à **moderniser** sa base technique.

2. La mission : 3 chantiers

Chantier 1 - Mise à jour & adaptation (*le cœur de la mission*)

1. **État des lieux des dépendances** : `npm audit`, `npm outdated` (ou équivalent de votre écosystème), repérage des versions **EOL / vulnérables / très en retard**.
2. **Montée de version** : mettez à jour plusieurs dépendances, dont **au moins une version majeure avec breaking change** *ou* **au moins une faille de sécurité corrigée**.
3. **Adaptation du code** : refactorisez le code **impacté** par ces montées de version (APIs changées, options dépréciées, signatures modifiées), en vous appuyant sur le **changelog / guide de migration** officiel de la lib.
4. **Preuve de non-régression** : un **filet de tests** garantit que le comportement n'a pas bougé.

Chantier 2 - Correctif

Corriger un **bug réel** : reproduire → diagnostiquer la **cause racine** → corriger → poser un **test de non-régression**.

Chantier 3 - Évolutif

Ajouter une **petite fonctionnalité** (ou faire évoluer une existante), testée.

En groupe : vous couvrez les 3 chantiers ; la charge est proportionnée à la taille de l'équipe (p. ex. un Chantier 1 plus ambitieux, ou un correctif/évolutif de plus par membre au-delà de 2). Le rendu indique **qui a fait quoi**.

3. La méthode imposée (le cœur de la note)

Travailler sur du legacy = **ne pas aggraver les choses**. Vous devez appliquer - et **prouver** :

- **Poser un filet avant de toucher.** Avant une montée de version ou un refacto, écrivez (ou lancez) des tests qui figent le comportement actuel ; après, ils doivent toujours passer.
 - **Lire le changelog.** Toute montée majeure s'accompagne d'une lecture du **guide de migration** : citez le ou les breaking changes que vous avez rencontrés.
 - **Reproduire avant de corriger.** Pas de fix à l'aveugle : on observe, on reproduit, on comprend.
 - **Prévoir le rollback.** Pour la montée de version : comment revenir en arrière si ça casse en prod ?
 - **Git propre.** Branche dédiée, **commits atomiques** (un commit = une intention), une **PR** avec description.
 - **Périmètre maîtrisé.** On ne mélange pas un upgrade, un bug et une feature dans le même commit.
-

4. Tableau de bord « avant / après » (obligatoire)

Mesurez l'état du projet **au début** et **à la fin** de la journée, et mettez les deux dans le rendu (**captures à l'appui**). C'est la preuve quantifiée de votre modernisation.

Indicateur	Avant	Après
Vulnérabilités (<code>npm audit</code>)		
Dépendances obsolètes (<code>npm outdated</code>)		
Build / lint OK ?		
Nb de tests / couverture (si dispo)		
(Front) score Lighthouse et (back) temps de réponse clé		

5. Fiche de cadrage

Chantier	Détail	« Fait » = quand...
C1 - Mise à jour	Dépendance(s) ciblée(s) + version actuelle → cible (préciser la majeure visée) ; plan de rollback	
C2 - Correctif	Bug ciblé + fichier(s)	
C3 - Évolutif	Fonctionnalité + fichier(s)	

Un upgrade purement cosmétique (patch sans impact) ou un chantier irréaliste (réécriture complète) n'est pas autorisé.

6. Le livrable : le compte-rendu de mission

Un **document unique** (PDF de préférence). Chaque chantier suit le même canevas, **preuves à l'appui**. Le rendu doit **regorger de captures d'écran** : c'est ce qui matérialise le travail.

Section	Contenu attendu	Captures / preuves obligatoires
0. Page de garde	Projet, équipe (+ rôle de chacun), lien dépôt, lien PR .	-
1. Reprise en main	État du projet au retour, temps pour le faire tourner , frictions (doc, .env , deps cassées).	Capture du projet qui démarre
2. Tableau de bord avant	Le tableau du §4, colonne « Avant ».	Captures : npm audit , npm outdated
3. Fiche de cadrage	Le tableau du §5, validé.	-
4. C1 - Mise à jour & adaptation	Deps montées (avant→après), breaking changes rencontrés (avec extrait du changelog), code adapté, plan de rollback .	Diff avant/après + extrait du changelog + capture des tests au vert
5. C2 - Correctif	Symptôme → reproduction → cause racine → correction → test.	Diff + test + capture avant/après du comportement
6. C3 - Évolutif	Besoin → implémentation → test.	Diff + capture de la fonctionnalité
7. Tableau de bord après + hygiène Git	Colonne « Après » + historique de la branche / PR.	Captures : npm audit après, historique Git / PR
8. Bilan & rétro legacy	Ce que les vieilles deps / le vieux code ont coûté ; ce que vous changeriez dans vos habitudes (tests, mises à jour régulières, doc) pour que votre futur-vous galère moins.	-

Captures d'écran obligatoires (récapitulatif)

- **npm audit** avant et après (le gain de sécurité).
- **npm outdated** avant (l'ampleur du retard).
- Les **tests au vert** après chaque chantier.
- **Avant/après** du correctif (comportement) et de l'évolutif (la feature).

- **Historique Git / PR.**
- **Le projet qui tourne.**

Dans toutes les captures et extraits : **secrets / clés / mots de passe masqués.**

Journal de bord (annexe attendue)

Un **log horodaté** de votre journée : « 10h12 - upgrade X 4→5, build cassé sur l'import Y → lecture du guide de migration → remplacé par Z ». Ça montre votre démarche réelle, y compris les impasses. C'est valorisé.

7. Grille d'évaluation (/20)

Critère	Points
Méthode legacy : filet avant de toucher, changelog lu, reproduction, Git/PR propre , plan de rollback	6
Chantier 1 - Mise à jour & adaptation : montée de version réelle (dont une majeure/faille) + refacto d'adaptation + non-régression prouvée + gain quantifié	5
Chantier 2 - Correctif : cause racine + test de non-régression	3
Chantier 3 - Évolutif : fonctionne, testé, intégré proprement	2
Qualité des preuves : tableau avant/après, captures, diffs, historique Git	2
Journal de bord & rétro : démarche réelle assumée, lucidité sur sa dette	2
Bonus : montée de version particulièrement épineuse bien gérée, filet exemplaire, doc utile laissée	+1

Repères. En dessous de la moyenne : un `npm update` lancé sans comprendre, un build cassé laissé tel quel, un fix à l'aveugle, des commits fourre-tout. Au-dessus : un mainteneur qui **sécurise avant d'agir, lit les changelogs, prouve** par des tests et des mesures que le projet est plus sain qu'au matin.

8. Conseils & pièges à éviter

- **Une majeure suffit.** Mieux vaut **une** montée de version majeure bien digérée (breaking changes compris et corrigés) que dix patchs anonymes.
 - **Lisez le guide de migration AVANT** de lancer l'upgrade : vous saurez quoi casser.
 - **Pas de refacto sans filet** : sans test qui fige l'existant, vous ne pouvez rien prouver.
 - **Ne corrigez jamais sans avoir reproduit** le bug.
 - **Commitez petit et souvent**, un chantier à la fois - un rollback propre vous sauvera.
 - Construisez le rendu **au fil de l'eau** (captures au moment où vous les faites).
-

9. Modalités de rendu

À rendre :

1. Le **compte-rendu** PDF.
 2. Le **lien de la PR** (accès lecture @celianlb).
-

Checklist

- ☐ Les **3 chantiers** sont traités et la **fiche de cadrage** est dans le rendu.
- ☐ **C1** : au moins une **montée de version majeure ou une faille corrigée**, avec **refacto d'adaptation** et **changelog cité**.
- ☐ **Tableau de bord avant/après** rempli, avec captures `npm audit` / `npm outdated`.
- ☐ Chaque chantier a un **diff avant/après** et des **tests au vert** (captures).
- ☐ Le correctif a **reproduction + cause racine + test de non-régression**.
- ☐ **Plan de rollback** documenté pour la montée de version.
- ☐ **Historique Git / PR** visible : branche dédiée, commits atomiques.
- ☐ **Journal de bord** présent.
- ☐ **Aucun secret** visible dans les captures/extraits.
- ☐ Fichier **nommé correctement** + **lien PR** accessible.

☐ (Groupe) La **contribution de chacun** est indiquée.